



Write Once
Run Anywhere

Exception Handling in Java

Handle Errors, Build Robust Applications

✉ rgupta.mtech@gmail.com

Same Language
Infinite Possibilities! 😊

1 What is Exception?

- Exception** is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
- Exception handling** is a convenient way to handle errors.

Examples of situations that cause exceptions:

- Attempt to divide an integer by zero.
- Attempt to call a method using a reference that is null.
- Attempting to open a nonexistent file for reading.
- JVM running out of memory.

2 Normal Flow vs Exception Flow



Exception handling do not correct abnormal condition rather it make our program robust i.e. make us enable to take remedial action when exception occurs...
Help in recovering...

3 Types of Exceptions

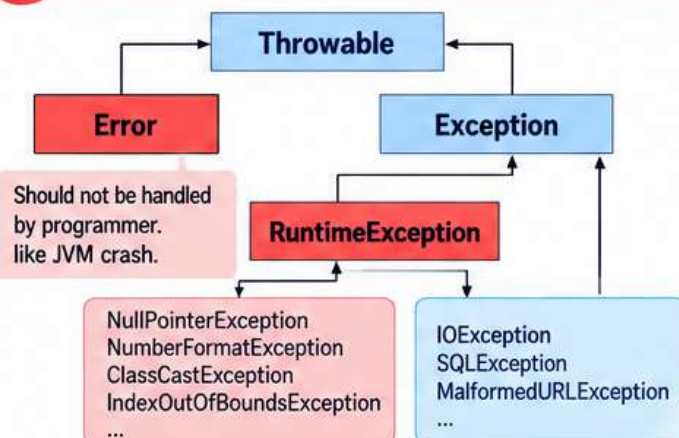
- 1 Unchecked Exception**
Also called Runtime Exceptions
- 2 Checked Exception**
Also called Compile time Exceptions
- 3 Error**
Should not to be handled by programmer.. like JVM crash

Key Keywords

1. try
2. catch
3. throw
4. throws
5. finally

All exceptions happens at run time in programming and also in real life.....
for checked ex, we need to tell java we know about those problems for example readLine() throws IOException

4 Exception Hierarchy



5 Checked vs Unchecked vs Error

Type	Also called	When to handle
Unchecked Exception	Runtime Exceptions	Not required to handle (compiles without handling)
Checked Exception	Compile time Exceptions	Must be handled (tell Java using try-catch or throws)
Error	-	Should not be handled by programmer (like JVM crash)

6 Simple Code Examples

Exception example (divide by zero)

```
public class Example1 {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        try {
            int c = a / b; // exception
            System.out.println(c);
        } catch (Exception e) {
            System.out.println("Error: " + e);
        } finally {
            System.out.println("Finally block");
        }
    }
}
```

Checked exception example

```
import java.io.*;

public class Example2 {
    public static void main(String[] args)
        throws IOException {
        BufferedReader br = new
            BufferedReader(new
                FileReader("test.txt"));
        String line = br.readLine();
        System.out.println(line);
    }
}
```



Exceptions are not failures, they are opportunities to handle problems better!

Better Code
Happier Developers! 😊

1 What is Exception?

- ❖ **Exception** – is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
- ❖ **Exception handling** is convenient way to handle errors
- ❖ Attempt to divide an integer by zero causes an exception to be thrown at run time.
- ❖ Attempt to call a method using a reference that is null. Attempting to open a nonexistent file for reading.
- ❖ JVM running out of memory.



- ❖ Exception handling do not correct abnormal condition rather it make our program robust i.e. make us enable to take remedial action when exception occurs...Help in recovering...

2 Type of exceptions

1. Unchecked Exception Also called Runtime Exceptions

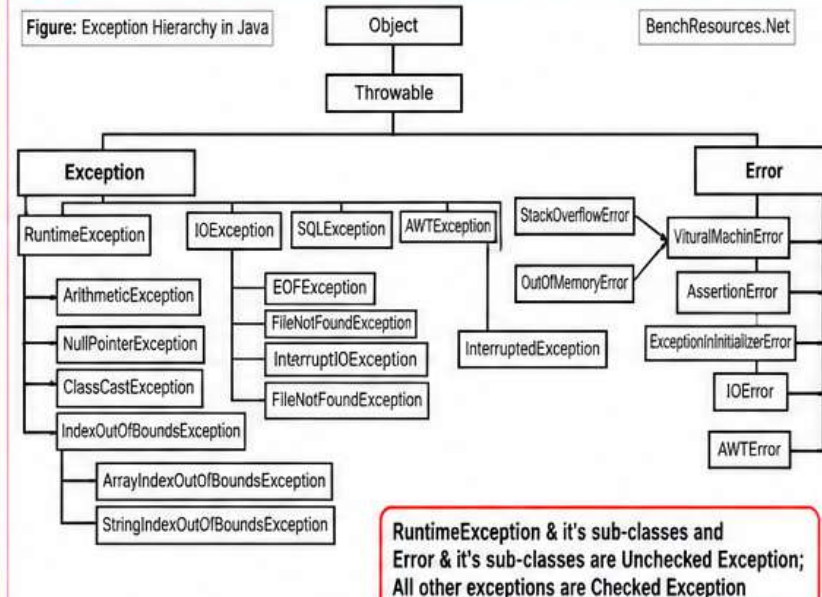
2. Checked Exception Also called Compile time Exceptions

1. try
2. catch
3. throw
4. throws
5. finally

3. Error Should not to be handled by programmer..like JVM crash

All exceptions happens at run time in programming and also in real life....
for checked ex, we need to tell java we know about those problems for example
readLine() throws IOException

3 Exception Hierarchy



4 Common Java Exceptions

- ArithmeticException
- ArrayIndexOutOfBoundsException
- ArrayStoreException
- FileNotFoundException
- IOException – general I/O failure
- NullPointerException – referencing a null object
- OutOfMemoryError
- SecurityException – when applet tries to perform an action not allowed by the browser's security setting.
- StackOverflowError
- StringIndexOutOfBoundsException

5 Defining new exception

If you create your own exception class from *RuntimeException*, it's not necessary to write exception specification in the procedure.

```

class MyException extends RuntimeException { }

public class ExampleException {
    static void doSomething(int n) {
        throw new MyException( );
    }

    public static void main(String[ ] args) {
        DoSomething(-1); // try / catch do not use
    }
}
  
```

6 Stack Trace

The exception keeps being passed out to the next enclosing block until:

- a suitable handler is found;
- or there are no blocks left to try and the program terminates with a stack trace

If no handler is called, then the system prints a stack trace as the program terminates

- it is a list of the called methods that are waiting to return when the exception occurred
- very useful for debugging/testing

The stack trace can be printed by calling printStackTrace()

```

// The getMessage and printStackTrace methods
public static void main( String[] args) {
    try {
        method1();
    } catch (Exception e) {
        System.err.println(e.getMessage() + "\n");
        e.printStackTrace();
    }
}
  
```



Java File I/O – Using Files, Creating Directories & Buffered Streams

Work with files and directories using java.io | Read and write data efficiently using buffered streams



1 File: using file, creating directories

File abstraction that represent file and directories

- ▶ File f = new File("....");
- ▶ boolean flag = file.createNewFile();
- ▶ boolean flag = file.mkdir();
- ▶ boolean flag = file.exists();

```
public class DemoFileWriter {  
    public static void main(String[] args) {  
        try {  
            BufferedWriter bw= new BufferedWriter  
                (new FileWriter("demo5.txt"));  
  
            bw.write("java is key");  
            bw.flush();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

2 Buffered Reader and BufferedWriter

Reading from console

```
BufferedReader br = new BufferedReader(  
    new InputStreamReader(System.in));
```

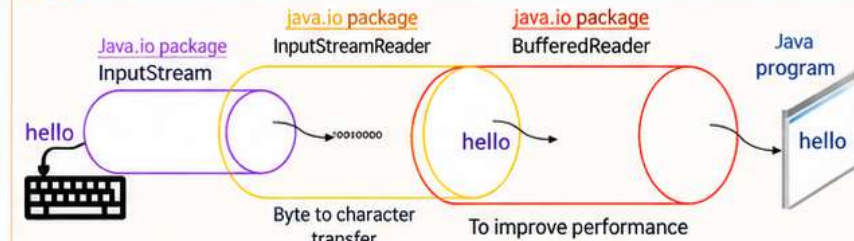
Reading from file

```
BufferedReader br = new BufferedReader(  
    new FileReader(new File("c:\\raj\\foo.txt")));
```

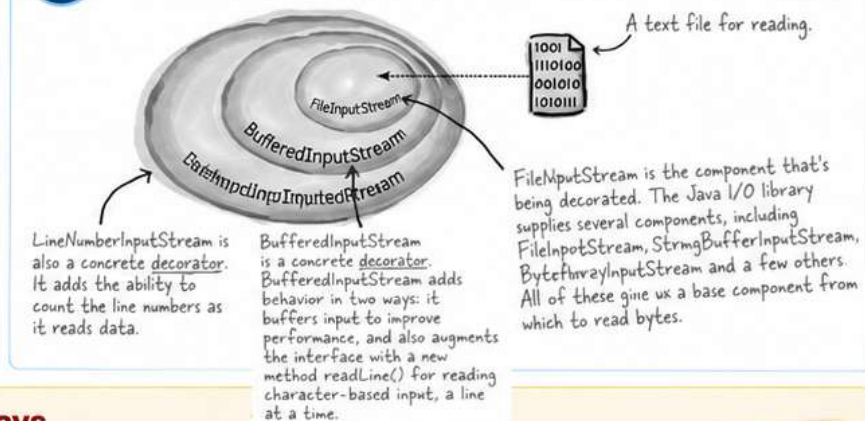
3 Reading file using BufferedReader (Example)

```
try(BufferedReader br = new BufferedReader(  
    new FileReader(new File(pathname: "data.txt"))){  
    String line=null;  
    while ((line=br.readLine())!=null) {  
        String [] tokens= line.split( regex: " ");  
        for(String token : tokens){  
            System.out.println(token);  
        }  
    }  
} catch (FileNotFoundException ex){  
    ex.printStackTrace();  
} catch (IOException ex){  
    ex.printStackTrace();  
} catch (Exception ex){  
    ex.printStackTrace();  
}
```

4 Byte to Character Flow with BufferedReader



5 Stream Decorators (Example with FileInputStream)



6 What are BufferedReader and BufferedWriter?

BufferedReader

- ▶ Reads text efficiently using a buffer
- ▶ Can be used with InputStreamReader (System.in) for console input
- ▶ Can be used with FileReader for file input
- ▶ Common method: readLine()

BufferedWriter

- ▶ Writes text efficiently using a buffer
- ▶ Can be used with FileWriter for file output
- ▶ Common methods: write(), flush()

Key Takeaways

- ▶ Use File to represent files and directories (createNewFile(), mkdir(), exists()).
- ▶ Use BufferedReader and BufferedWriter to improve performance.
- ▶ BufferedReader can read from console (System.in) or a file.
- ▶ BufferedReader readLine() reads a line at a time and can be split into tokens.
- ▶ Streams can be layered (decorated) to add additional functionality.

Use
Buffered Streams
for better
performance!

Java I/O Streams

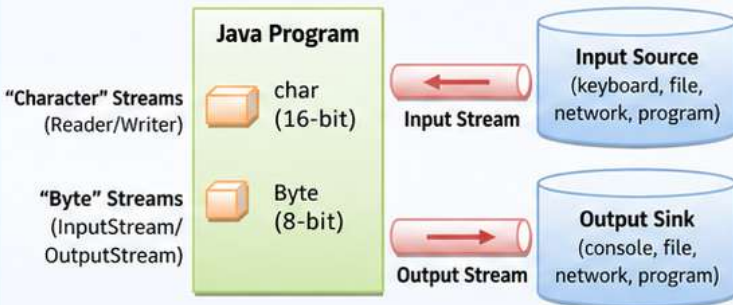
READ | PROCESS | WRITE | USING **java.io**

Streams Connect
Your Program
with the Real World



1 Stream I/O in Standard I/O (java.io Package)

Streams in java.io are used to read data from an input source and write data to an output sink.



Internal Data Formats:

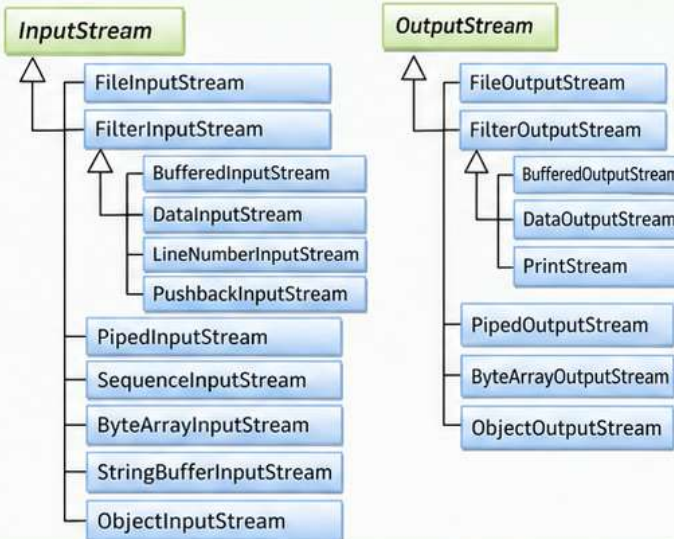
- Text (char): UCS-2
- int, float, double, etc.

External Data Formats:

- Text in various encodings (US-ASCII, ISO-8859-1, UCS-2, UTF-8, UTF-16, UTF-16BE, UTF16-LE, etc.)
- Binary (raw bytes)

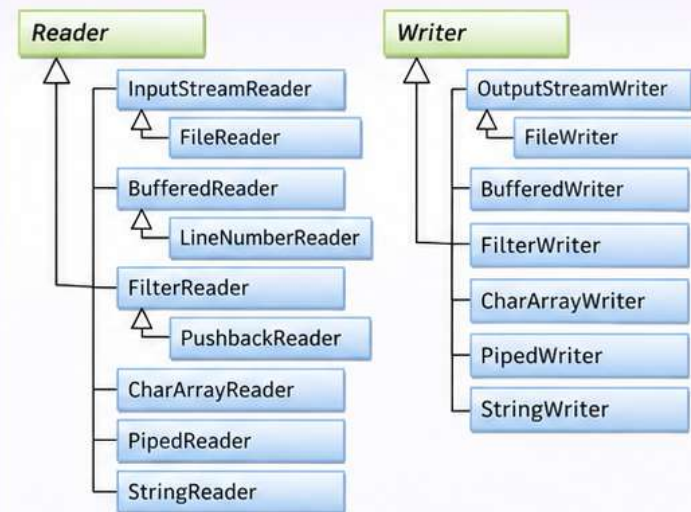
2 Byte-Based I/O & Byte Streams

Class hierarchy for byte-based streams (InputStream / OutputStream) in java.io package.



3 Character-Based I/O & Character Streams

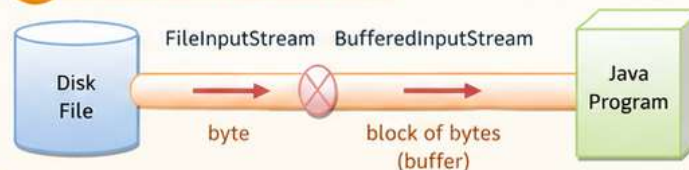
Class hierarchy for character-based streams (Reader / Writer) in java.io package.



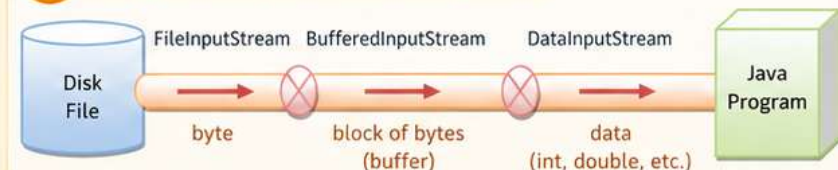
4 Layered (or Chained) I/O Streams

Streams can be layered (chained) to provide additional functionality such as buffering and data conversion.

4a Using BufferedInputStream



4b Using BufferedInputStream and DataInputStream





Java I/O and Performance

Same File Copy Operation – Different Streams, Different Performance

Stream Chaining
Improves Performance



1 Copying a photo without buffer

```
//time taken: 807 ms
FileInputStream fis=new FileInputStream(
    name: "/home/raj/Desktop/photo/mali.jpg");
FileOutputStream fos=new FileOutputStream(
    name: "/home/raj/Desktop/photo/mali_copy.jpg");

long start=System.currentTimeMillis();

int byteRead=1;
while ((byteRead=fis.read())!=-1){
    fos.write(byteRead);
}

fis.close();
fos.close();

long end=System.currentTimeMillis();
System.out.println("time taken: " + (end-start)+" ms");
```



2 Copying a photo with our own buffer

```
//time taken: buffer
//time taken: 1 ms
FileInputStream fis=new FileInputStream(
    name: "/home/raj/Desktop/photo/mali.jpg");
FileOutputStream fos=new FileOutputStream(
    name: "/home/raj/Desktop/photo/mali_copy.jpg");

long start=System.nanoTime();

byte[] byteBuffer=new byte[4*1024];

int numberOfByteRead=1;
while ((numberOfByteRead=fis.read(byteBuffer))!=-1){
    fos.write(byteBuffer, 0, numberOfByteRead);
}

fis.close();
fos.close();

long end=System.nanoTime();
System.out.println("time taken: " + (end-start)+" ms");
```



3 Copying a photo with BufferedInputStream

```
BufferedInputStream fis=new BufferedInputStream(
    new FileInputStream( name: "/home/raj/Desktop/photo/mali.jpg"));

BufferedOutputStream fos=new BufferedOutputStream(
    new FileOutputStream( name: "/home/raj/Desktop/photo/mali_copy.jpg"));

long start=System.currentTimeMillis();

int byteRead=1;
while ((byteRead=fis.read())!=-1){
    fos.write(byteRead);
}

fis.close();
fos.close();

long end=System.currentTimeMillis();
System.out.println("time taken: " + (end-start)+" ms");
```



Performance Comparison

Without buffer
(FileInputStream + FileOutputStream)

Time taken: 807 ms

With our own buffer
(byte[] of 4*1024)

Time taken: 1 ms

With Buffered Streams (Stream Chaining)
BufferedInputStream + BufferedOutputStream

Significantly faster
(uses internal buffer)



Key Takeaway

Streams can be chained (layered) to add buffering and improve performance. Using BufferedInputStream and BufferedOutputStream is a simple and effective way to get better I/O performance for file operations.

*Same code logic
Better performance
with stream chaining!*